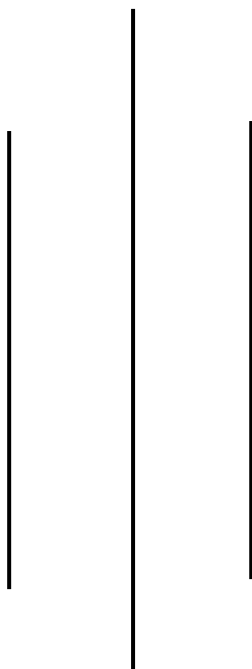


NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University



LAB REPORT FOR EMBEDDED SYSTEMS



LAB REPORT 4: VHDL CODE FOR ADDER

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20 th Nov	
2.	VHDL Code (OR & NOT Gate) using testbench	23 rd Nov	
3.	Universal Gates and Special Gates	30 th Nov	
4.	VHDL Code for Adder	7 th Dec	

LAB 4: VHDL CODE FOR ADDER

OBJECTIVE:

The objective of this lab is to understand the basic concept of binary addition by designing and implementing **Half Adder** and **Full Adder** circuits using **VHDL**. This lab aims to familiarize students with combinational logic design, Boolean expressions of adders, and VHDL coding for digital arithmetic circuits. It also helps in simulating and verifying the correctness of adder outputs using different input combinations.

THEORY:

Introduction to Adder

An adder is a **combinational digital circuit** that performs the addition of binary numbers. Adders are fundamental components used in arithmetic logic units (ALUs), processors, calculators, and digital systems. Based on the number of input bits, adders are mainly classified into **Half Adder** and **Full Adder**.

Half Adder

A **Half Adder** is a digital circuit that adds **two single-bit binary inputs** and produces two outputs:

- **Sum (S)**
- **Carry (C)**

It does not consider any carry from a previous stage, hence the name *half* adder.

Inputs

- A
- B

Outputs

- Sum (S)
- Carry (C)

Logic Equations

- **Sum (S) = $A \oplus B$**
- **Carry (C) = $A \cdot B$**

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half adders are simple but cannot be used alone for multi-bit addition because they do not handle carry-in from previous bits.

Full Adder

A **Full Adder** is an advanced combinational circuit that adds **three single-bit inputs**:

- Two significant bits
- One carry input from the previous stage (**Cin**)

It produces two outputs:

- **Sum (S)**
- **Carry Out (Cout)**

Inputs

- A
- B
- Cin

Outputs

- Sum (S)
- Carry Out (Cout)

Logic Equations

- **Sum (S) = $A \oplus B \oplus \text{Cin}$**
- **Carry (Cout) = $(A \cdot B) + (B \cdot \text{Cin}) + (A \cdot \text{Cin})$**

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A full adder can be implemented using **two half adders and an OR gate**, making it suitable for cascading to form multi-bit adders.

Adder Implementation Using VHDL

VHDL (VHSIC Hardware Description Language) is used to describe the behavior and structure of digital circuits. In this lab, VHDL is used to model half adder and full adder circuits using **concurrent signal assignments** and **Boolean logic expressions**. Simulation of the VHDL code helps in verifying the correct operation of adders for all possible input combinations.

VHDL CODE FOR HALF ADDER:

Using Data Flow:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity ArpHalfAdder is
port(
  A, B : in Std_logic;
  sum, carry : out Std_logic
);
end ArpHalfAdder;
architecture archHalfAdder of ArpHalfAdder is
begin
  sum <= A xor B;
  carry <= A and B;
end archHalfAdder;
```

Using Behavioral:

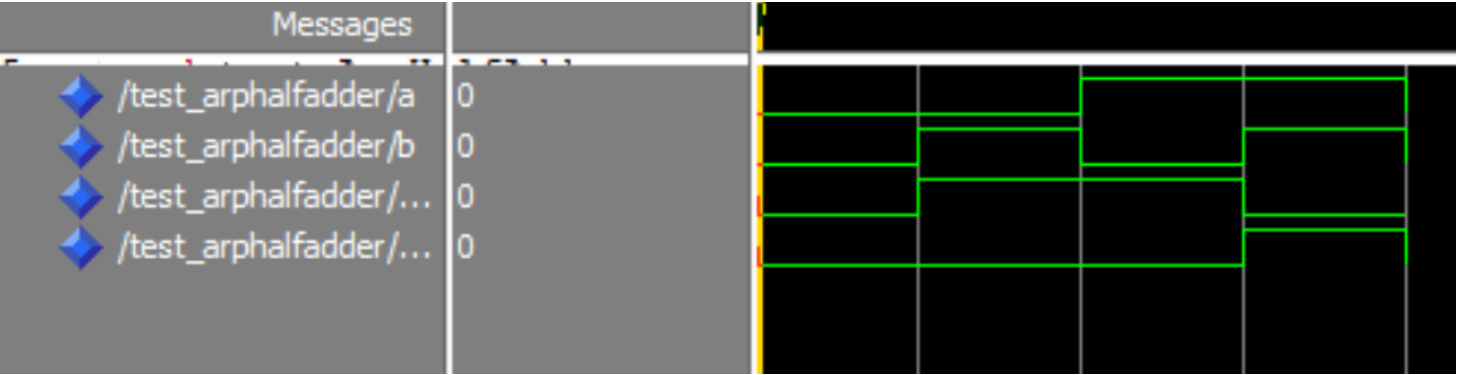
```
library IEEE;
use IEEE.std_logic_1164.all;
Entity ArpHalfAdder is
port(
  A, B : in Std_logic;
  sum, carry : out Std_logic
);
end ArpHalfAdder;
architecture archHalfAdder of ArpHalfAdder is
begin
  process(A,B)
  begin
    if (A = '0' and B = '0') then
      sum <= '0';
      carry <= '0';
    elsif ((A = '1' and B = '0') or (A = '0' and B = '1')) then
      sum <= '1';
      carry <= '0';
    else
      sum <= '0';
      carry <= '1';
    end if;
  end process;
end archHalfAdder;
```

TEST BENCH FOR HALF ADDER:

```
library IEEE;
use IEEE.std_logic_1164.all;
entity test_ArpHalfAdder is
end test_ArpHalfAdder;
architecture archTest of test_ArpHalfAdder is
component ArpHalfAdder
    port(
        A, B : in Std_logic;
        sum, carry : out Std_logic
    );
end component;
Signal A,B : std_logic := '0';
Signal sum, carry : std_logic;
begin
    UUT : ArpHalfAdder port map(
        A => A,
        B => B,
        sum => sum,
        carry => carry
    );

    stim_proc : process
    begin
        A <= '0'; B<= '0';
        wait for 100 ps;
        A <= '0'; B<= '1';
        wait for 100 ps;
        A <= '1'; B<= '0';
        wait for 100 ps;
        A <= '1'; B<= '1';
        wait for 100 ps;
    end process;
end archTest;
```

Waveform for Half Adder:



VHDL CODE FOR FULL ADDER:

USING DATA FLOW:

```
library IEEE;
use IEEE.std_logic_1164.all;

Entity ArpFullAdder is
port(
    A, B, Cin : in Std_logic;
    sum, carry : out Std_logic
);
end ArpFullAdder;

architecture archFullAdder of ArpFullAdder is
begin
    sum <= A xor B xor Cin;
    carry <= (A and B) or (B and Cin) or (A and Cin);
end archFullAdder;
```


USING BEHAVIORAL:

```
library IEEE;
use IEEE.std_logic_1164.all;

Entity ArpFullAdder is
port(
    A, B, Cin : in Std_logic;
    sum, carry : out Std_logic
);
end ArpFullAdder;

architecture archFullAdder of ArpFullAdder is
begin
    process(A, B, Cin)
    begin
        if (A='0' and B='0' and Cin='0') then
            sum <= '0';
            carry <= '0';
        elsif ((A='0' and B='0' and Cin='1') or
            (A='0' and B='1' and Cin='0') or
            (A='1' and B='0' and Cin='0')) then
            sum <= '1';
            carry <= '0';
        elsif ((A='0' and B='1' and Cin='1') or
            (A='1' and B='0' and Cin='1') or
            (A='1' and B='1' and Cin='0')) then
            sum <= '0';
            carry <= '1';
        else
            sum <= '1';
            carry <= '1';
        end if;
    end process;
end archFullAdder;
```

TEST BENCH FOR FULL ADDER

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_ArpFullAdder is
end test_ArpFullAdder;

architecture archTest of test_ArpFullAdder is
component ArpFullAdder
    port(
        A, B, Cin : in Std_logic;
        sum, carry : out Std_logic
    );
end component;

Signal A, B, Cin : std_logic := '0';
Signal sum, carry : std_logic;

begin

    UUT : ArpFullAdder port map(
        A => A,
        B => B,
        Cin => Cin,
        sum => sum,
        carry => carry
    );

    stim_proc : process
    begin
        A <= '0'; B <= '0'; Cin <= '0';
        wait for 100 ps;
        A <= '0'; B <= '0'; Cin <= '1';
        wait for 100 ps;
        A <= '0'; B <= '1'; Cin <= '0';
        wait for 100 ps;
        A <= '0'; B <= '1'; Cin <= '1';
        wait for 100 ps;
        A <= '1'; B <= '0'; Cin <= '0';
        wait for 100 ps;
        A <= '1'; B <= '0'; Cin <= '1';
        wait for 100 ps;
        A <= '1'; B <= '1'; Cin <= '0';
        wait for 100 ps;
        A <= '1'; B <= '1'; Cin <= '1';
        wait for 100 ps;
    end process;

end archTest;
```

RESULT:

The Half Adder and Full Adder circuits were successfully designed and implemented using **VHDL**. Both **data flow** and **behavioral** modeling techniques were used, and the designs were verified using appropriate **test benches**. The simulation waveforms showed correct **Sum** and **Carry** outputs for all possible input combinations, matching the respective truth tables of Half Adder and Full Adder.

CONCLUSION:

This practical helped in understanding the working of **binary adders** and their implementation using **VHDL**. Through this experiment, the difference between **Half Adder and Full Adder** was clearly observed, especially the role of carry input in the Full Adder. The use of data flow and behavioral models improved understanding of different VHDL coding styles. The successful simulation confirms that VHDL can be effectively used to design and verify basic arithmetic circuits used in digital systems.